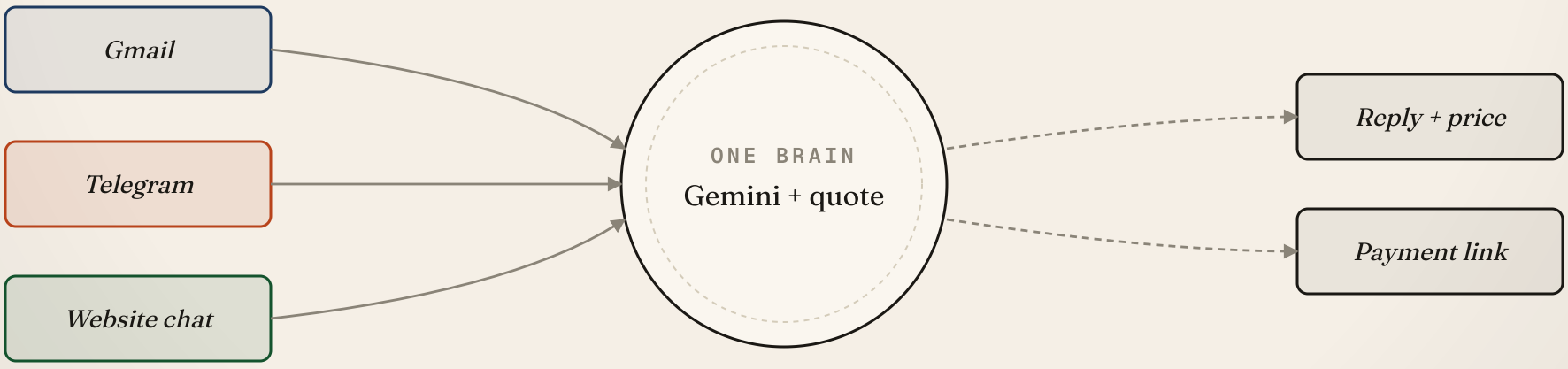
BUILT THIS WEEK · N8N + GEMINI · NO SERVER TO RUN

My bakery's website chatbot takes *real orders.* I never ran a server.



THREE WAYS IN · ONE PLACE TO THINK · THE RIGHT WAY BACK OUT

— THE FUNDAMENTAL

> *A channel is just a door. Build one brain, then add doors.*

Most teams rebuild the whole assistant for every new channel — a bot for the website, another for WhatsApp, a third for email. **That's three brains to keep in sync.** The durable move is the opposite: one place that thinks, and thin adapters that carry messages in and out.

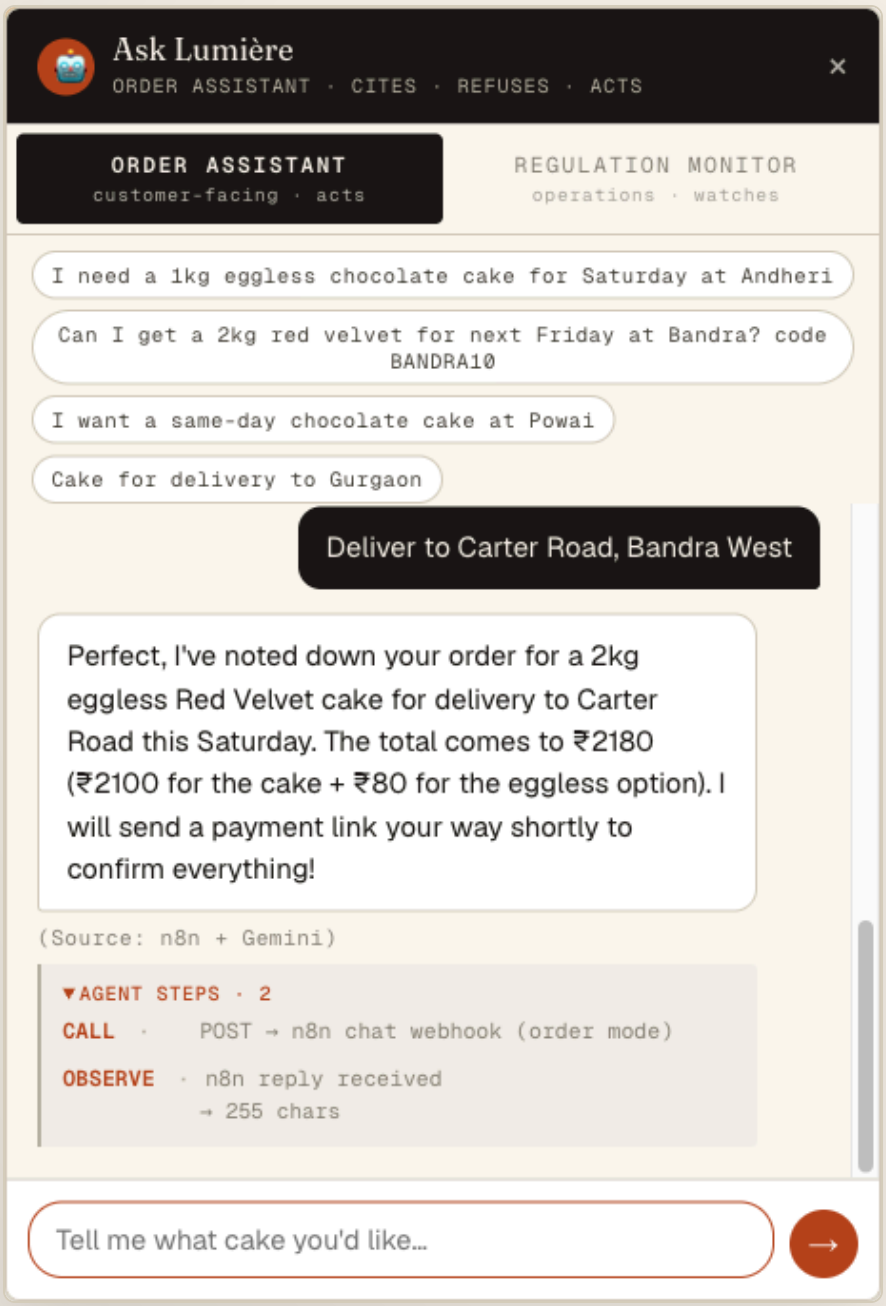
— THE MECHANISM · 4 MOVES

Tag at the door. Decide at the exit.

1	A message arrives — email, Telegram DM, or website chat	PERCEIVE
2	Stamp it with *where it came from* — one field: *channel*	TAG
3	Same Gemini prompt reads the order; the quote API prices it	THINK
4	Read the tag → reply on the channel it *came in on*	ACT

The brain never learns what a "channel" is.
Two lines of config do all the routing.

— LIVE CONVERSATION · WEBSITE CHANNEL



WHAT'S ACTUALLY HAPPENING

THE ASK

2kg eggless red velvet, Saturday, Carter Road

THE REPLY

Order confirmed · *₹2,180* · ₹2,100 cake + ₹80 eggless

THE RECEIPT

POST → n8n chat webhook · reply in 255 chars

WHAT IT TRACKED

Pulled the order together *from the thread*, then priced it

It held the order across the conversation and did the maths in reply — **none of it hard-coded.**

SWIPE →

— THE BUILD · ONE WORKFLOW

Four nodes carry the website chat.

WEBSITE CHAT PATH · ADDED ALONGSIDE GMAIL & TELEGRAM

IN	THINK	PRICE	OUT
Webhook	Gemini	Quote API	Respond

Browser posts the message + the chat history → **Gemini** *reads the menu and the thread* → *reply goes straight back to the browser.* **Memory comes from passing the history every turn.**

The site is a static page on Vercel. The "backend" is a serverless function for pricing.
The intelligence lives in n8n — the page just talks to it.

— THE PATTERN, GENERALISED

Adding a channel is adding a door.

✉	**Gmail** *customer emails an order*	LIVE
✈	**Telegram** *customer DMs the bakery bot*	LIVE
💬	**Website chat** *visitor types on the site*	LIVE
+	**WhatsApp · Instagram · a form** *whatever's next*	SAME BRAIN

Every new channel is a trigger + an adapter.
The thinking never gets rewritten.

1

— MOVE 1 · TAG AT THE ENTRANCE

Stamp the channel the moment it arrives.

SET NODE · RIGHT AFTER EVERY TRIGGER

```
channel = "web"     // or "gmail", "telegram"
// keep all the original fields too
```

One field, set once per door. Everything downstream reads it instead of guessing where the message came from.

Without the tag, your reply node has to reverse-engineer the source. With it, routing is a lookup.

2

— MOVE 2 · GIVE MEMORY FOR FREE

Pass the whole thread, every turn.

WHAT THE BROWSER SENDS TO THE WEBHOOK

```
{ "message": "2kg of that, eggless",
  "history": [ ...previous turns ] }
```

The model has no memory of its own. You get continuity by handing it the conversation each time — so "that" still means the red velvet from two messages ago.

Stateless model + stateful client = a chat that remembers, with zero database.

3

— MOVE 3 · SHIP IT WHERE IT CAN'T DIE

Static page. Serverless price. Hosted brain.

No always-on server means nothing to crash at 2am. The site is a file; the price is a function that wakes on demand; the agent runs in n8n. **Each piece scales to zero.**

INBOUND CHANNELS	ALWAYS-ON SERVERS	WEB REPLY	BRAINS TO SYNC
3	0	~4s	1
Gmail · Telegram · web	*nothing to babysit*	*webhook round-trip*	*one prompt, all doors*

THE TAKEAWAY

Pick the brain first. The *channels* are easy.

If you're rebuilding your assistant for every new surface, you're maintaining the same thing three times. Build it once. Add doors.

✦ *I teach non-coders to build agents like this. What channel would you wire up first — and what would it do? Tell me below.*

Shantanu Chandra

Building and teaching applied AI agents · Generative AI for Non-Coders

in

linkedin.com/in/chandrashantanu

FOLLOW FOR THE FIELD GUIDE